

PRE-CLASS READING 4 OF 5

Agents as a product — Hassabis, Gemini Spark and the universal assistant

Hassabis, D. (2026). Google I/O keynote and press remarks, May 2026 — the launch of **Gemini Spark**, Google's always-on personal agent, together with his stated position on agents, world models and AGI timelines. Read as the commercial counterpart to Readings 1–3: what happens when an agent stops being an engineering pattern and becomes a US\$20-a-month product for hundreds of millions of people.

ASSIGNED READING

Read **one** of the I/O 2026 write-ups linked at the end (the Fast Company interview piece is the most substantive on Hassabis's own arguments), plus the Gemini Spark product explainer for what the thing actually does and what it costs. Optional: the DeepMind "Release Notes" podcast episode on world models, for his view on where the next capability jump comes from. (≈ 25 min)

» [source ~25 min](#) • [this summary + reasoning notes ~18 min](#)

Note on currency. This is the most fast-moving item in the set. The facts below were correct as at **mid-August 2026**: Spark launched at I/O in May 2026 on the Google AI Ultra tier (repriced to US\$100/month, with a US\$200 tier above it), and was extended to the US\$20 AI Pro tier on **27 July 2026**, with a wider international rollout for Ultra subscribers that excludes the EEA, UK, Switzerland and Nigeria. Assume tiers, prices and availability have moved again by the time you read this — and note that this is itself the lesson: agent pricing is unsettled because nobody yet knows what an agent costs to run.

KEY TAKEAWAY

Readings 1–3 argue about whether agents are the right architecture. Google shipped one to consumers anyway. **Gemini Spark** is a persistent, cloud-resident agent that keeps running when your device is off: it watches Gmail, manages Calendar, drafts in Docs, runs scheduled and conditionally triggered tasks, chains steps across apps, reaches third-party services (Canva, OpenTable, Instacart) through the Model Context Protocol (MCP), and learns reusable "**teachable skills**" from natural-language instructions. Hassabis's framing is that this is the beginning of a **universal assistant** rather than a chatbot with extras, that AGI arrives "by 2030, plus or minus a year", and that agency has always been the point — "AlphaGo was an agent." **The three things to carry past this week are not the features.** They are these:

- **The price is the cost envelope.** Nobody publishes what an agent costs to run; a subscription price plus a usage cap is the closest public proxy — and the path from US\$100 to US\$20 in ten weeks is a statement about direction, not just about marketing.
- **The permission model is the product.** An agent's capability is exactly the set of tools it has been granted, so what it may touch is what it is — and the guardrail lives in the grant, not in the prompt.
- **The safety story is now a consumer-facing feature.** SynthID watermarking, C2PA content credentials, and a design that is "designed to ask you first" before spending your money — sold as reasons to buy, not filed as compliance.

WHY IT MATTERS FOR THIS COURSE

Capsule 3 asks when an agent is worth its token bill. A subscription price is the most legible answer anyone has published: whatever an always-on agent costs Google to run, they are betting it averages under the monthly fee — and the fact that Spark launched at US\$100 and reached US\$20 within ten weeks tells you something about the direction of inference cost that no benchmark does. Capsule 2 asks how you harden an agent; Spark is a live demonstration of hardening as **product design** rather than as prompt engineering — permissions off by default, explicit confirmation before high-stakes actions, a bounded set of connected services. And for your A2, Spark is the closest public reference for the thing you are building: an agent with tools, a cost model and a guardrail checklist. It is also a useful antidote to the framing in Reading 3 — Hassabis agrees world models matter, and is building them **inside** a shipping product rather than instead of one.

Feeds Class 4: C1 (what a deployed agent actually looks like) • C2 (permissions and confirmation as guardrails) • C3 (subscription pricing as the cost envelope • the A2 reference design).

MAIN POINTS

Product facts, prices and quotations are as reported by the sources named at the end. Anything after Our reading is this brief's interpretation — the cost arithmetic and the "two explanations" framing are ours, not Google's.

What Spark is, mechanically

- **Always-on, server-side.** Unlike a chat session, Spark runs on Google's infrastructure whether or not your device is awake. That single design choice is what makes scheduled and triggered work possible — "draft a reply when

this person emails”, “build me a cited digest every morning” — and it is also what makes the cost unpredictable, because the user is no longer the rate limiter.

- **Tools, by connector.** It acts across Gmail, Calendar and Docs, and reaches outside Google through **MCP** connectors to third parties. Note the architecture: the agent's power is bounded by the set of tools it has been granted, which is exactly the ACI point from Reading 1 arriving as a consumer permission screen.
- **Teachable skills.** You describe a repeated task once in natural language and it becomes a reusable routine. *Why this matters:* it is the workflow/agent distinction from Reading 1, exposed to end users — the first run is agentic and exploratory; the saved skill is a workflow, and cheaper.
- **Model and speed.** Reported to run on Gemini 3.5 Flash, positioned as outperforming the previous Pro-tier model on agentic tasks while being substantially faster. The pattern to notice: the agent tier runs on a **fast, cheap** model, not the biggest one. Latency and cost per step dominate when there are many steps.
- **Bounded on purpose.** US-only at launch, permissions disabled by default, user-controlled app access, explicit confirmation before high-stakes actions such as spending money, and a limited set of connected services. Every one of those is a guardrail that lives in the product, not in the prompt.

Hassabis's own arguments

- **Agents are not new to DeepMind.** “AlphaGo was an agent” — his consistent line is that DeepMind came to language models from reinforcement learning and agency, not the other way round. Useful corrective to the assumption that agents are a 2024 invention that emerged from chatbots.
- **AGI by 2030, ±1 year.** Stated plainly at I/O 2026. Treat it as a well-informed opinion from an interested party, not a forecast with a methodology — and contrast it with LeCun's “much harder than people think” from Reading 3. The interesting question for class is not who is right but **what each would have to see to update.**
- **Breadth over expertise.** “The sweet spot is to help everyone with these agents” — the target is not developers. That is a strategy statement with an engineering consequence: an agent for everyone cannot assume the user can debug it, so the reliability bar and the confirmation design have to carry the load that a technical user's judgement would otherwise carry.
- **Security as a differentiator.** He publicly criticised a rival agent product as insecure — “I wouldn't recommend it for any real work.” Whatever you make of the specific claim, note the market signal: with agents, **safety is being sold**, not merely complied with.
- **Provenance — what made this.** Google is pushing **SynthID** watermarking (cited at 60 years of video and 100 billion images watermarked) and supporting the **C2PA** content-credentials standard. The two are complementary bets: a watermark lives inside the content and survives stripping but needs the issuer's detector; a content credential travels alongside the file and anyone can verify it, but platforms routinely strip it. Neither says whether the content is *true* — only what produced it. An agent sharpens this because it generates and publishes without a human approving each artifact.
- **Misuse — what this helps someone do.** A separate problem, and watermarking is irrelevant to it. Hassabis named chemical, biological, radiological and nuclear (**CBRN**) **uplift** as his near-term concern — not whether a model can discuss chemistry, but whether it gives someone a marginal capability gain over a library and a search engine. His proposed mitigations are usage monitoring and **chain-of-thought inspection**: reading the reasoning trace to catch intent before the action runs, which works only while that reasoning stays legible (Reading 2). An agent sharpens this too — it does not just describe a step, it can take one. Both halves are Week 6 material arriving early, from a vendor.

The economics hiding in the price list

- **The launch price was a cost signal.** Spark arrived on a tier that had been US\$249.99/month a year earlier and was cut to US\$100 at I/O 2026, with a US\$200 tier above it for higher usage limits. Ten weeks later it reached the US\$20 Pro tier. Whatever the marketing, the sequence tells you the unit economics moved fast — and that Google was willing to find out in public what an always-on agent costs.
- **Usage limits are the real pricing mechanism.** The tiers differ not in features so much as in **how much the agent may do** (roughly 5× and 20× the Pro allowance). *What to take from this:* when the product is a loop, you cannot price per seat — you price per unit of work, and then hide it behind a tier so the customer does not have to think about tokens. Your A2 cost model faces exactly the same problem in miniature.
- **Free-tier absence is informative.** Spark is not on the free tier or on AI Plus. Agents are the first Google AI feature in years that could not be given away — the clearest available evidence that per-user inference cost for agentic workloads is still material.
- **The Pareto problem, returning from Week 3.** Class 3's AI Wars case established that a small minority of users generate most of the queries. An always-on agent makes this worse, because a heavy user's agent works overnight. Usage caps are not a nicety; they are the only thing standing between a flat subscription and an unbounded liability.

READING FOCUS QUESTIONS — bring your answers to class

There are no wrong answers here — these surface how you think, not what you recall. A clue sits under each question to give you a direction to push on; the fuller reasoning is in the section after.

- An always-on agent at US\$20/month: sketch the arithmetic that has to hold for that to work, and say which single number you would most want to know.

Clue. Do the arithmetic before you form a view. Pick a plausible task cost — say a multi-step task costing 20,000 tokens all-in — and work out how many tasks a month the fee buys at a few plausible prices per million tokens. Then ask what the distribution across users does to your answer, not just the average. The number you end up wanting is almost never “cost per token”.

- Spark’s guardrails are permissions-off-by-default, a bounded connector list, and confirmation before high-stakes actions. Map each one onto the failure it prevents — and name one failure none of the three catches.

Clue. Work through the layers rather than judging the list as a whole: one limits what the agent can reach, one limits what it may do with what it reaches, and one inserts a human at the moment of consequence. Then look for the failure that survives all three — the honest candidate is the action that is fully permitted, individually harmless, and wrong. That is the one that needs an eval, not a permission.

- Hassabis says AGI by 2030 ± 1 ; LeCun (Reading 3) says the whole approach is a dead end. Both are running well-funded labs. What can you actually do with expert disagreement of this size when you have to make a decision this quarter?

Clue. Resist picking a side — the useful move is to find the decisions that are robust either way. Ask which of your choices would change if you knew who was right: your architecture probably would; your evals, your guardrails, your data hygiene and your ability to swap models almost certainly would not. Sort your decisions into those two piles and the disagreement stops being paralysing.

- Spark launched at US\$100/month and reached US\$20 in about ten weeks. Give two competing explanations for that — one about cost, one about strategy — and say what evidence would separate them.

Clue. Two stories fit the same fact: inference got cheaper, or Google decided distribution was worth more than margin. They make different predictions about everything else you can observe — usage caps, rate limits, model choice, whether rivals follow, whether the price holds. Write down what each story predicts before you decide which you believe; that is the same falsification habit Reading 3 asked for.

- Your A2 team is building an agent prototype. Which two design decisions from Spark would you copy outright, and which one would you deliberately not?

Clue. Judge each decision by what it would cost you to build and what it buys per unit of cost — a confirmation gate is nearly free and removes your worst outcome; an always-on scheduler is expensive and buys you very little in a two-week project. Also ask which decisions only make sense at Google’s scale. Copy the cheap risk-removers; skip the ones whose value depends on having a hundred million users.

HOW TO THINK ABOUT THESE — read after you have had a go

These are not model answers to be memorised — they are the line of reasoning we will pick up in class. Class time is short, so they are written to stand on their own.

Q1 — The arithmetic, and the number that actually matters

Our reading — none of this arithmetic is published; it is an order-of-magnitude estimate. Start with a rough task: a multi-step agent task consuming on the order of 20,000 tokens once you count the resent context (Reading 2’s quadratic effect). At around US\$0.30 per million tokens for a fast model, that is about US\$0.006 a task, so a US\$20 subscription buys on the order of three thousand such tasks a month before gross margin disappears — comfortable, until you notice that a background agent with scheduled triggers can consume that on its own without the user typing anything. Which is why the number you actually want is not cost per token or even cost per task; it is **cost per user per month at the 95th percentile**. Averages are useless here because usage is Pareto-distributed — Week 3’s AI Wars case put the shape on it — and a flat fee against a long-tailed cost distribution is a bet that the tail is short. Everything in Google’s pricing follows from that: tiers defined by usage multiples rather than features, caps rather than unlimited, a fast cheap model rather than the flagship, and no free tier at all. When you build your A2 cost model, model the heavy user, not the median one; the median user never breaks a business.

Q2 — Three layers, and the failure that survives them

Permissions-off-by-default limits the **blast radius**: an agent that cannot reach your bank cannot empty it, whatever it decides. The bounded connector list limits the **action space**: fewer tools, fewer ways to be creative in the wrong direction, and — from Reading 1 — a smaller, cleaner tool set the model chooses from more accurately. Confirmation before high-stakes actions inserts a **human at the point of irreversibility**, which is the exact remedy Reading 3 pointed at: you cannot simulate the consequence, so you make a person approve it. Together they cover most catastrophic outcomes. What they do not catch is the **permitted, individually harmless, cumulatively wrong**

action: the agent has Gmail access, is allowed to draft and send routine replies, and sends fifty polite, fluent, subtly incorrect answers to customers overnight. No permission was violated and no confirmation was warranted for any single one. That failure class is only caught by evaluation and monitoring — sampling outputs, judging them against sources, watching for drift — which is precisely why Reading 1's eval section and Week 6's observability material exist. Permissions bound the worst case; evals catch the ordinary case going quietly wrong at scale.

Q3 — Building on contested ground

Two of the most credentialed people in the field disagree about whether the current approach can get to general intelligence, and you have a project due. The way through is to sort decisions into **architecture-dependent** and **architecture-independent**. Architecture-dependent: how much autonomy you grant, whether you invest in long-horizon planning, whether you bet a product on reliability that does not exist yet. Those genuinely hinge on who is right, so make them reversible and revisit them. Architecture-independent — and this is most of what you will actually do: writing evals that describe what good looks like; instrumenting so you can see what happened; putting guarantees in code rather than prompts; keeping the model layer swappable behind an interface; getting your data and permissions in order. None of those change if world models arrive in 2029, and all of them get more valuable. There is also a synthesis available: Hassabis is not on the other side of LeCun's argument so much as on a different timeline of it — DeepMind is building world-model capability *into* shipping products rather than waiting for a clean architecture. If both labs are betting on world models and disagreeing only about whether today's systems are a stepping stone or a detour, then the practical implication for you this quarter is close to nil, and you should notice that before the debate consumes your planning meeting.

Q4 — Cost story or strategy story

Our reading. The cost story: inference for agentic workloads got dramatically cheaper — a faster small model, better caching, better routing — and the price followed. The strategy story: Google decided that owning the default assistant on a billion devices is worth more than margin on it, and priced for distribution while a rival was slower. Both fit the observed fact, so look at what else they predict. The cost story predicts caps loosening over time, the same capability appearing on cheaper tiers, competitors following within a quarter, and the price holding. The strategy story predicts caps staying tight or tightening, aggressive bundling, the agent being pushed hardest where it defends an existing franchise (search, Workspace), and prices that could rise once adoption is locked in. Watch the usage limits rather than the headline price — limits are where the true unit cost shows through, because marketing sets prices and finance sets caps. This is the same analytical move as Class 3's AI Wars case: the sticker price tells you about competition, the unit economics tell you about the business, and only one of the two appears in the press release.

Q5 — What to copy into A2, and what to leave

Copy the **confirmation gate on irreversible actions** — it costs an afternoon, removes your worst demo outcome, and is the single clearest thing you can point to in the guardrail checklist your A2 has to produce. Copy the **bounded, explicit tool set with permissions off by default**: it makes your system easier to reason about, easier to evaluate, and it directly improves tool-selection accuracy, so it buys reliability as well as safety. Do not copy **always-on background execution**. It is the most expensive property in the product, its value comes from persistence over months, and in a two-week project it converts straightforwardly into an unbounded token bill and a debugging nightmare — a manually triggered or scheduled-once run demonstrates the same capability at a fraction of the risk. If you want a third to copy, take **teachable skills**, but in the humble form: when your agent solves something in six exploratory steps, write those steps down as a fixed workflow and run *that* next time. You will have converted an agent into a workflow, which is where Reading 1 started, and you will be able to show the cost difference on a slide — which is precisely what ILO 3 asks you to be able to justify.

APPENDIX — PRODUCT AND ECONOMICS TERMS, EXPLAINED

The distinction that carries this reading: an agent as an engineering pattern versus an agent as a priced product. The moment it becomes a product, its cost, its permissions and its failure modes all become someone's P&L.

Always-on / persistent agent

- **Plain.** An agent that keeps working on Google's servers when your laptop is shut.
Technically. Server-side execution with scheduled and conditionally triggered runs. Decouples workload from user attention — which is the feature, and also the reason cost per user becomes hard to forecast.

Teachable skill

- **Plain.** Describe a repeated task once; the agent saves it as a routine you can re-run.
Technically. User-authored, reusable task definition. Structurally, the promotion of an exploratory agent run into a workflow (Reading 1) — cheaper, more predictable, and testable in a way the exploratory version is not.

Model Context Protocol (MCP)

- **Plain.** A common plug standard so an agent can talk to outside services without bespoke code for each.

Technically. Open protocol for exposing tools and data sources to models; used by Spark for third-party connectors (e.g. Canva, OpenTable, Instacart). Turns “what can this agent do?” into a question about which servers are connected.

Usage limits / tiering

- **Plain.** You buy an allowance of agent work, not a set of features.

Technically. Spark’s tiers differ chiefly by usage multiple (roughly 5× and 20× the Pro allowance). The standard answer to pricing a loop under a flat fee: cap the work, since cost scales with steps taken rather than seats sold.

Confirmation gate

- **Plain.** The agent stops and asks before doing something you cannot undo.

Technically. “Designed to ask you first” for high-stakes actions such as spending money. A human-in-the-loop control placed at the point of irreversibility — the practical substitute for being unable to simulate consequences in advance (Reading 3).

SynthID / C2PA

- **Plain.** Invisible watermarks in generated media, plus a standard label saying where a file came from.

Technically. SynthID embeds detectable watermarks in generated images, audio and video (Google cites 60 years of video and 100 billion images); C2PA Content Credentials is a cross-industry provenance standard. Both are provenance controls, not capability controls — they say what made a thing, not whether it is true.

CBRN uplift

- **Plain.** The worry that a model meaningfully helps someone build a chemical, biological, radiological or nuclear weapon.

Technically. The dominant near-term catastrophic-risk category in frontier-lab safety frameworks; Hassabis’s stated one-to-two-year concern, with monitoring and chain-of-thought inspection as proposed mitigations. Returns in Week 6.

SOURCE LINKS

Fast Company — Hassabis at Google I/O 2026 · www.fastcompany.com/91544235/demis-hassabis-google-io-2026 (the most substantive on his own arguments)

Gemini Spark — product explainer · www.datacamp.com/blog/gemini-spark (what it does, tiers, limits)

Spark extended to the US\$20 AI Pro tier, 27 July 2026 · chromeunboxed.com/googles-agent-ai-gemini-spark-is-now-available-on-the-20-pro-tier-in-the-us/

DeepMind Release Notes podcast — Hassabis on world models ·

blog.google/innovation-and-ai/models-and-research/google-deepmind/ai-release-notes-podcast-demis-hassabis/ (optional)

Companion · **Reading 3** (LeCun) is the architectural objection to what Spark does; **Reading 5** (Devin) is what the same product bet looked like two years earlier, and how it was judged.

CAVEATS

1. This summary is built from press coverage and product explainers, not from a primary Google publication. Quotes are as reported; treat specifics — prices, tiers, model names, availability — as correct at mid-August 2026 and likely stale by the time you read this.
2. Reported figures differ slightly between outlets on the Ultra tier structure (US\$100 and US\$200 variants). Where sources conflict, the brief says so rather than picking one.
3. Hassabis is describing and selling his own company’s product. The AGI timeline, the security criticism of a rival, and the “universal assistant” framing are all positions taken by an interested party.
4. This summary was generated and fact-checked using Anthropic models — a competitor of the company described here. The cost arithmetic, the three-layer guardrail reading and the two-explanations framing are the instructor’s. Flag anything that looks wrong.